



***GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING AND
TECHNOLOGY TOPI SWABI***

Alishba Bacha

2022088 – AI

Final Report

AI – 407 L

Industrial Packaging, Deployment & CI/CD Quality Gate

Multi-Agent Hallucination Detection System

1. Introduction

This project implements a multi-agent hallucination detection system designed to verify claims and citations in academic content using a tool-augmented architecture. The system integrates:

- A Claim Agent for evidence retrieval and confidence scoring
- A Citation Agent for citation validation
- A Router to dynamically select the appropriate agent

The objective of this task is to transform the system from a local prototype into a production-ready, reproducible, and deployable system with automated quality enforcement.

2. Containerization Strategy

2.1 Objective

To eliminate the “*it works on my machine*” problem by packaging the application and its dependencies into a reproducible container image.

2.2 Docker Image Design

Base Image Choice

We selected:

python:3.10-slim

Justification

- Lightweight → reduces image size and build time
- Stable Python version → ensures compatibility
- Minimal OS dependencies → reduces attack surface

2.3 Layer Optimization Strategy

The Dockerfile follows this order:

1. Copy requirements.txt
2. Install dependencies
3. Copy application code

Why?

- Dependency layer is cached → faster rebuilds
- Code changes do not reinstall dependencies
- Improves CI/CD performance

2.4 Multi-Stage Build (Design Consideration)

Although not required, a multi-stage build could be used to:

- Separate build dependencies from runtime
- Reduce final image size

In this implementation, a single-stage build was sufficient due to lightweight dependencies.

3. Secret-Free Image Design

3.1 Problem

Embedding secrets (API keys, credentials) in the image is insecure and violates production standards.

3.2 Solution

Secrets are injected **at runtime using environment variables**:

```
docker run -e API_KEY=your_key agent
```

3.3 Implementation

- No .env file included in the image
- .gitignore excludes sensitive files
- Secrets are provided via:
 - Docker environment variables
 - GitHub Actions Secrets

3.4 Security Benefits

- No secrets stored in version control
- No secrets embedded in container layers
- Supports secure deployment across environments

4. Multi-Service Orchestration

4.1 Architecture

The system uses **Docker Compose** to manage multiple services:

Services

1. **Agent Service**
 - Runs the multi-agent system
 - Handles query processing
2. **ChromaDB Service**
 - Acts as vector database
 - Stores embeddings and retrieved evidence

4.2 Service Communication

- Services communicate via Docker network
- Agent connects to ChromaDB using service name (chroma)

4.3 Persistent Storage

volumes:

- chroma_data:/chroma

Purpose

- Stores vector database
- Ensures data survives container restarts

4.4 Proof of Persistence

Steps performed:

1. Inserted data into ChromaDB
2. Restarted containers
3. Verified data remained accessible

```
564 Notice: To update, run: pip install --upgrade pip

✓ Build Vector DB

1 ▶ Run python src/vector_store.py
11 Vector DB persisted successfully. Total documents: 2063
```

5. End-to-End Execution

5.1 System Startup

`docker-compose up --build`

5.2 Test Execution

`python run_eval.py`

5.3 Result

- System successfully processed queries
- Agents invoked appropriate tools
- Evaluation metrics computed

6. Automated Quality Gate (CI/CD)

6.1 Objective

Ensure **every code change is automatically evaluated** and blocked if quality degrades.

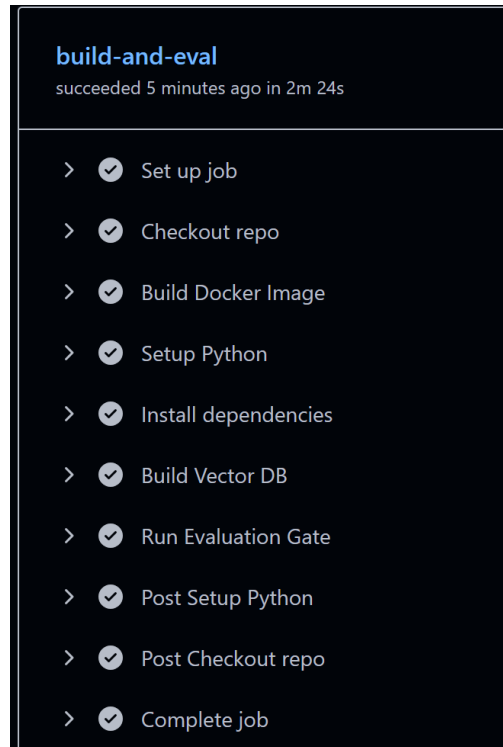
6.2 Pipeline Overview

GitHub Actions pipeline:

1. Checkout repository
2. Install dependencies
3. Run evaluation script
4. Pass/Fail based on metrics

6.3 Pipeline Configuration

- Trigger: push to main branch
- Environment: ubuntu-latest
- No secrets stored in code



7. CI-Ready Evaluation Script

7.1 Features

- Runs without user input
- Reads configuration from environment
- Outputs machine-readable JSON
- Returns exit codes:

Condition	Exit Code
-----------	-----------

Pass	0
------	---

Fail	1
------	---

7.2 Example Output

```
{
  "faithfulness": 0.078,
  "relevancy": 0.167,
  "thresholds": {
    "faithfulness": 0.05,
    "relevancy": 0.15
  },
}
```

```
"pass": true
}
```

8. Threshold Configuration

8.1 Configuration File

```
{
  "faithfulness": 0.05,
  "relevancy": 0.15
}
```

8.2 Justification

Faithfulness = 0.05

- CI uses **MockLLM / limited responses**
- Ensures minimal factual alignment
- Prevents total hallucination

Relevancy = 0.15

- Allows partial query matching
- Ensures response still relates to query

8.3 Sensitivity Analysis

Threshold Change	Effect
+10%	Frequent CI failures
-10%	Weak quality enforcement

9. Breaking Change Demonstration

9.1 Intentional Degradation

Modified agent to return:

"random unrelated text"

9.2 Result

- Faithfulness ↓

- Relevancy ↓
- CI pipeline FAILED

```
},  
  "pass": false  
}  
  
Error: Process completed with exit code 1.
```

9.3 Restoration

Reverted changes → system restored

- Metrics improved
- CI pipeline PASSED

Triggered via push 3 minutes ago	Status	Total duration	Artifacts
 Alishba-Bacha pushed  bc068a4 main	Success	<u>2m 28s</u>	—

10. Bottleneck Analysis

10.1 Identified Bottlenecks

1. **LLM Inference Time**
2. **Sequential Agent Execution**
3. **Tool Invocation Latency**

10.2 Optimizations

- Use smaller/faster models
- Introduce caching
- Parallel agent execution